

Recurrent networks

Dataset: Chicago transit ridership

How to use these notes

The primary text for this lecture is Géron, Chapter 13. These notes are not a replacement for it and do not repeat it: they are the lecture’s own argument written out at length — the derivation done slowly, the numbers with the runs that produced them, and the reasoning between one slide and the next that a deck can only gesture at. Read the chapter for the method; read these for why the lecture makes the choices it does. Every figure quoted here is printed by this lecture’s notebook, and the course checks that mechanically rather than trusting it.

Contents

1	What a linear model on lags cannot do	2
1.1	Unrolling, and the problem it creates	2
2	Four improvements, measured	3
2.1	Deeper — which failed	3
2.2	More series — which worked	4
2.3	Further ahead	4
2.4	Gates	5
3	The whole application	6
3.1	Red-team it: what happens when the world changes	6
4	What recurrence still cannot do	7
5	The notebook	8
6	Exercises	8
	Summary	8

1 What a linear model on lags cannot do

Lecture 15 established the ground rules and a number to beat: evaluation is forward in time, never a random split; the seasonal-naive forecast is genuinely hard to beat and the autocorrelation says why; and a linear model on lagged features beats it, modestly.

That model has a *fixed window*. Anything outside the last 56 days is invisible to it, and every lag gets one weight regardless of context. Take 365 lags and it has 365 weights, each fitted from the same amount of data, each learning one number about one position — and the weight on lag 7 learns nothing from the weight on lag 14, though both are “a week ago”.

A recurrent network keeps a *state* instead, and that state is not bounded by a window:

$$\mathbf{h}_t = \phi\left(\mathbf{W}_x^T \mathbf{x}_t + \mathbf{W}_h^T \mathbf{h}_{t-1} + \mathbf{b}\right).$$

This is Lecture 9’s $\phi(\mathbf{XW} + \mathbf{b})$ with one extra term, and that term is the whole idea. $\mathbf{W}_x$ and $\mathbf{W}_h$ do not depend on t : one set of weights applied at every step, so reach is decoupled from parameter count — the same argument as weight sharing in Lecture 12, along time instead of across an image.

The state is a summary, and that is a budget

$\mathbf{h}_t$ is everything the network has decided to remember about $x_1, \dots, x_t$, at *fixed width* whatever the sequence length — so it is lossy by construction. What it keeps is learned, not designed: nothing tells the cell that “the same weekday last week” matters.

Two sequences with the same state are indistinguishable to everything downstream, which is the definition of a sufficient statistic, and the network is trying to learn one. It is also the honest limit: if the task needs more bits than the state has, no amount of training supplies them.

1.1 Unrolling, and the problem it creates

Draw the same cell once per time step and you get a network 56 layers deep with one set of weights, reused. Then differentiate it exactly as Lecture 10 taught — **backpropagation through time** needs nothing new.

Failure condition — one matrix raised to the length of the sequence

It is multiplied by the same factor 56 times. Here the factor is *the same matrix* every step, because the weights are shared — so the gradient is governed by a power of one matrix, and powers of a matrix either vanish or explode unless the spectral radius sits almost exactly at one.

Initialisation cannot fix this the way it did in Lecture 11, where the layers were independent. Something has to change in the cell itself.

That is also why tanh rather than ReLU: a bounded activation keeps the repeated product from exploding.

The model is eleven lines — an `nn.RNN` and a linear head applied to `outputs[:, -1]`, the hidden state after the final step. The head is not optional: the state has 32 components and the target has one, and tanh outputs lie in $(-1, 1)$ while our scaled targets sometimes exceed that. `batch_first=True` because our batches are `[batch, time, series]`; without it the module silently reads the first dimension as time.

The training loop is Lecture 10's five lines, unchanged. The loss is Huber — quadratic near zero, linear far away — because we are optimising a proxy for MAE, and minimising the absolute error directly gives a gradient that never gets smaller as you approach the optimum. *The metric and the training loss are allowed to differ*; Lecture 2 said so, and here is the case.

Shuffle, but shuffle the right thing

`DataLoader(train_set, shuffle=True)` shuffles the *windows*. It does not shuffle the values inside a window — the sequence within each row is untouched.

Consecutive windows share almost all their content: one starting on day t and one on day $t + 1$ differ in two values out of fifty-six. Without shuffling, a mini-batch is fifty near-identical examples, its gradient is one example's gradient with less noise, and training stalls.

And do not let that shuffling cross the split. Windows are shuffled within the training period only. Shuffling and leakage are different operations, and it is easy to conflate them.

On identical shuffled folds the recurrent network scores 38,224, against 55,385 for copying last week and 44,925 for the linear model — 31.0% better than the baseline and 15% better than the linear model. Both beat the baseline by more than the fold-to-fold spread, so the ordering is real, and the recurrent model wins because it can use the *order* of the window rather than a weighted sum of it.

Failure condition — and the assumption, stated rather than left implied

Those folds are shuffled, so *none of those numbers is a forecast*. Lecture 15 measured what that is worth. They are comparable with each other because the folds are identical, and with nothing else.

Every forward number in the rest of this lecture is on the honest ladder.

2 Four improvements, measured

2.1 Deeper — which failed

One argument, `num_layers=3`. The simple RNN forecasts at 49,073 and the three-layer one at 53,491.

It got worse, and not from overfitting: its error on the *training* rows is higher too, 40,630 against 35,972. Three times the parameters fitted on the same 1,040 windows, and gradients now traversing 56 time steps *and* three layers — Lecture 11's vanishing signal compounded.

Report the negative result. A ladder of improvements in which every rung works is a ladder somebody edited.

2.2 More series — which worked

We have been forecasting rail from rail alone while a bus series sits unused, and a `day_type` column we established is known in advance. Adding both changes the architecture by one number, `input_size=5`.

Wait — that is a `shift(-1)`

Lecture 15 spent ten minutes on a random split that let a model train on rows surrounding what it was scored on. This one reads the future *deliberately*.

The test that separates them: *is the value available at prediction time?* The type of the day we are forecasting is on the public calendar and has been for a year. Its ridership is not.

This is the one question about leakage that no static check can answer for you. It is not a property of the code; it is a property of the world.

Model	Forecast MAE	vs copying last week
Copy last week	64,815	—
Simple RNN, rail only	49,073	24% better
Simple RNN, five inputs	40,046	38% better

Eighteen per cent off the error for one line of feature engineering and one changed constructor argument. Most of it is the day type: the network no longer has to infer that a holiday is coming from the shape of the previous eight weeks — which is impossible — because we told it.

2.3 Further ahead

Three strategies, all defensible, all to be evaluated forward in time at the horizon you actually need. **Recursive** predicts one step and feeds it back, so errors compound and the model never trained on its own output. **Direct** fits a separate model per horizon, so nothing is shared. **Sequence-to-sequence** emits k outputs at once.

A fourth variant predicts fourteen at *every* step, taking the head over all outputs rather than the last. That is not cheating: a recurrent cell at step t has seen steps 1 to t and nothing more — it is **causal**. The loss simply gains a term at every step, so many more gradients flow and they travel a shorter distance. After training, only the last step's output is used.

Measured, the error grows with distance from 35,997 at one day to 46,586 at fourteen, and stays below the no-model baseline throughout.

Teacher forcing, and what it costs

Feeding the decoder the true previous token trains fast and stable, but the model has never seen its own mistakes; feeding its own prediction matches deployment and makes early training noise.

Exposure bias is the mismatch: a model trained only on true prefixes meets a distribution at prediction time it never saw — its own errors, compounding — which is precisely the recursive strategy's weakness.

Failure condition — bidirectional layers, and the one question

Running a second layer backwards lets every position see the whole sequence. A backward pass *reads the future*: fine when the whole sequence is available at prediction time, and leakage when it is not.

A bidirectional forecaster is Lecture 15's random split wearing a different hat — the number goes up and the system cannot be deployed. Ask: at prediction time, do I have the rest of the sequence? For text classification, usually yes. For forecasting, never.

2.4 Gates

A plain cell multiplies the state at every step, so $\partial \mathbf{h}_t / \partial \mathbf{h}_{t-1} = \text{diag}(\phi') \mathbf{W}_h^T$ and over T steps the gradient carries T copies of one matrix.

What a gated cell does instead

An LSTM keeps a separate cell state updated *additively*,

$$\mathbf{c}_t = \mathbf{f}_t \odot \mathbf{c}_{t-1} + \mathbf{i}_t \odot \tilde{\mathbf{c}}_t.$$

When the forget gate is near one, $\partial \mathbf{c}_t / \partial \mathbf{c}_{t-1} \approx \mathbf{1}$ and the gradient passes through almost unchanged.

That is the entire trick: *a path along which the gradient is added to rather than multiplied.*

Gate	Form	What it decides
forget $\mathbf{f}_t$	sigmoid	how much of the old cell state to keep
input $\mathbf{i}_t$	sigmoid	how much of the candidate to write
candidate $\tilde{\mathbf{c}}_t$	tanh	what to write, if anything
output $\mathbf{o}_t$	sigmoid	how much of the cell state to expose

Every gate is a sigmoid of the same two inputs with its own weights, so a sigmoid in $[0, 1]$ is a *soft switch* and the network learns when to open it. A GRU merges forget and input into one update gate — what is kept is what is not written — and has no separate cell state, keeping the additive near-identity path with about three quarters of the parameters.

Model, all five inputs	Forecast MAE	Training MAE
Simple RNN	40,046	27,035
LSTM	37,542	24,540
GRU	34,639	20,356

The GRU wins, and the second column says why to be careful: it fits the training rows 25% better than the plain cell but the forecast only 14% better. That gap is overfitting, and it is the next thing to attack.

A one-dimensional convolution with stride 2 before the recurrent layer lets the same number of steps cover twice the history — and on this series it does not pay, 39,792 against 35,997 at one day and 55,148 against 46,586 at fourteen. Reported because we measured it, not because it won.

Gradient clipping is not optional here

The gradient through T steps carries a power of one matrix. When its spectral radius exceeds one even slightly the norm does not drift upwards — it explodes over a handful of steps, and a single bad batch can move every weight somewhere training never recovers from. Clip by *global norm*, not per parameter: the direction of the update is information and only its length is the problem.

3 The whole application

Forecast	What we reported	What it does
Copy last week	55,385	64,815
Linear, 56 lags	44,925	56,446
Simple RNN	38,224	49,073
GRU, five inputs	—	34,639

The two middle rows are the same models measured twice. The bottom row is the model we would deploy, and it was never measured the wrong way. *The final number is better than the one we committed to — but we reached it by first admitting that the committed one was fiction.*

Is 34,639 good? It is 47% better than a system a transit authority could run with no data scientist at all, and its typical miss is around 10.4% of the day it forecasts, against 16.5% for copying last week *on those same 151 days*. Lecture 15's 12.0% was the naive forecast over all 1,191 pool days, which is a different set of days and not the comparison to make here.

What a defensible report says

“Forecasting the five months after the training cut, and never seeing them, the model’s mean absolute error is 34,639 boardings a day against 64,815 for copying the same day of the previous week. Both figures are on identical rows. The gain is concentrated on holidays.” The protocol, the baseline, the rows, and where the gain comes from — four things the number alone does not carry.

3.1 Red-team it: what happens when the world changes

Fit on everything to the end of 2019 and forecast the four months from March 2020.

	MAE
The linear model, trained to the end of 2019	84,380
Copying last week, on the same 122 days	36,282
The actual mean level over that period	141,404

The model is 2.3 times worse than doing nothing, and its error is six tenths of the entire level.

Failure condition — and the shuffled split cannot see it at all

The same shuffled five-fold cross-validation over 2016–2021, spanning the collapse, reports 36,554 — *better* than before, because the post-2020 days are easy to interpolate: flat, low, and surrounded by other flat low days that are in the training set.

A random split does not merely overstate performance. It is *structurally incapable* of detecting a regime change, which is the failure that actually ends forecasting systems.

To be fair to the model: March 2020 is not a leak and not its fault. No forecaster could have produced it from eight weeks of history, and copying last week wins only because it adapts within a week of the collapse. The lesson is not that the model is bad — it is that *an honest protocol told you the model had failed, on the day it failed, and the dishonest one reported an improvement*. Which is why a deployed forecaster is monitored against its own naive baseline, continuously: if the margin disappears, something has changed in the world.

Temporal leakage checklist

1. Is the split by *date*, not by row?
2. Does every feature exist at the moment the forecast is made? Name the moment.
3. Were any statistics computed over the whole series before the cut?
4. Do the training windows overlap the scored windows? Purge them.
5. Is the baseline computed on identical rows to the model?
6. Does the reported score come from more than one origin?

The third is Lecture 2's leak meeting today's.

4 What recurrence still cannot do

It is *sequential*: step t needs step $t - 1$, so the computation cannot be parallelised along the sequence. Batching parallelises across sequences, never along one, so training time grows linearly with sequence length on any hardware. On a 56-step window nobody cares; on a document of ten thousand tokens it is the binding constraint, and it is the practical reason — more than the gradient — that attention replaced recurrence.

The state is a *bottleneck*: everything remembered must fit in one fixed-width vector. And long-range dependence is still hard — gates make the gradient survive, but they do not make position 400 as accessible as position 2.

Let every position look directly at every other, with learned weights, and all three problems go at once: the computation parallelises, there is no single state to squeeze through, and position 400 is exactly as far away as position 2. That is attention, and it is Lecture 18.

5 The notebook

What to change

Lengthen the window from 56 steps to 200 and re-run the plain RNN. It gets *worse*, not better, and the gradient probe — Lecture 11's, along time instead of depth — says why.

6 Exercises

1. Explain why unrolling a recurrent cell over T steps makes the gradient harder to control than a T -layer feed-forward network, and why initialisation cannot repair it. [5]
2. State the update of an LSTM's cell state, and explain in one sentence why its additive form is what lets the gradient survive. [4]
3. A three-layer recurrent network scores worse than a one-layer one on both the forecast *and* the training rows. State what that rules out, and what it suggests instead. [4]
4. A feature is built with `shift(-1)`. Give the single question that decides whether this is leakage, and explain why no static check can answer it. [4]
5. A shuffled cross-validation over a period spanning a regime change reports a better score than one that excludes it. Explain the mechanism, and what it implies about monitoring a deployed forecaster. [5]

Summary

A recurrent cell is Lecture 9's layer with one extra term, and that term decouples reach from parameter count — weight sharing along time instead of across an image. The state is a fixed-width lossy summary, which is a budget: if the task needs more bits than it holds, no amount of training supplies them. Unrolling makes training ordinary backpropagation on a 56-layer network with *shared* weights, and that sharing is the problem, because the gradient then carries a power of one matrix rather than a product of independent ones.

Four improvements, measured forward in time, and one of them failed: a three-layer network scored worse on the forecast *and* on the training rows, which rules out overfitting and points at optimisation. Adding the bus series and the next day's type — a `shift(-1)` that is legitimate because the calendar is public — took the error down 18%. Gates buy an additive path along which the gradient is added to rather than multiplied, and the GRU reached 34,639.

The ladder ends 47% better than copying last week, and the honest ladder is not the one we first reported: the two middle rows are the same models measured twice. Then the red team — a model trained to the end of 2019 is 2.3 times worse than doing nothing on the months from March 2020, while the shuffled protocol reports an *improvement* over that same period, because flat low days surrounded by flat low days are easy to interpolate. A random split does not merely overstate performance; it cannot see a regime change at all.